Algoritmo e Estrutura de Dados II
CTCO02

Balanceamento de árvores binárias
AVL – Inserção

Vanessa Souza

Universidade Federal de Itajubá – Instituto de Matemática e Computação

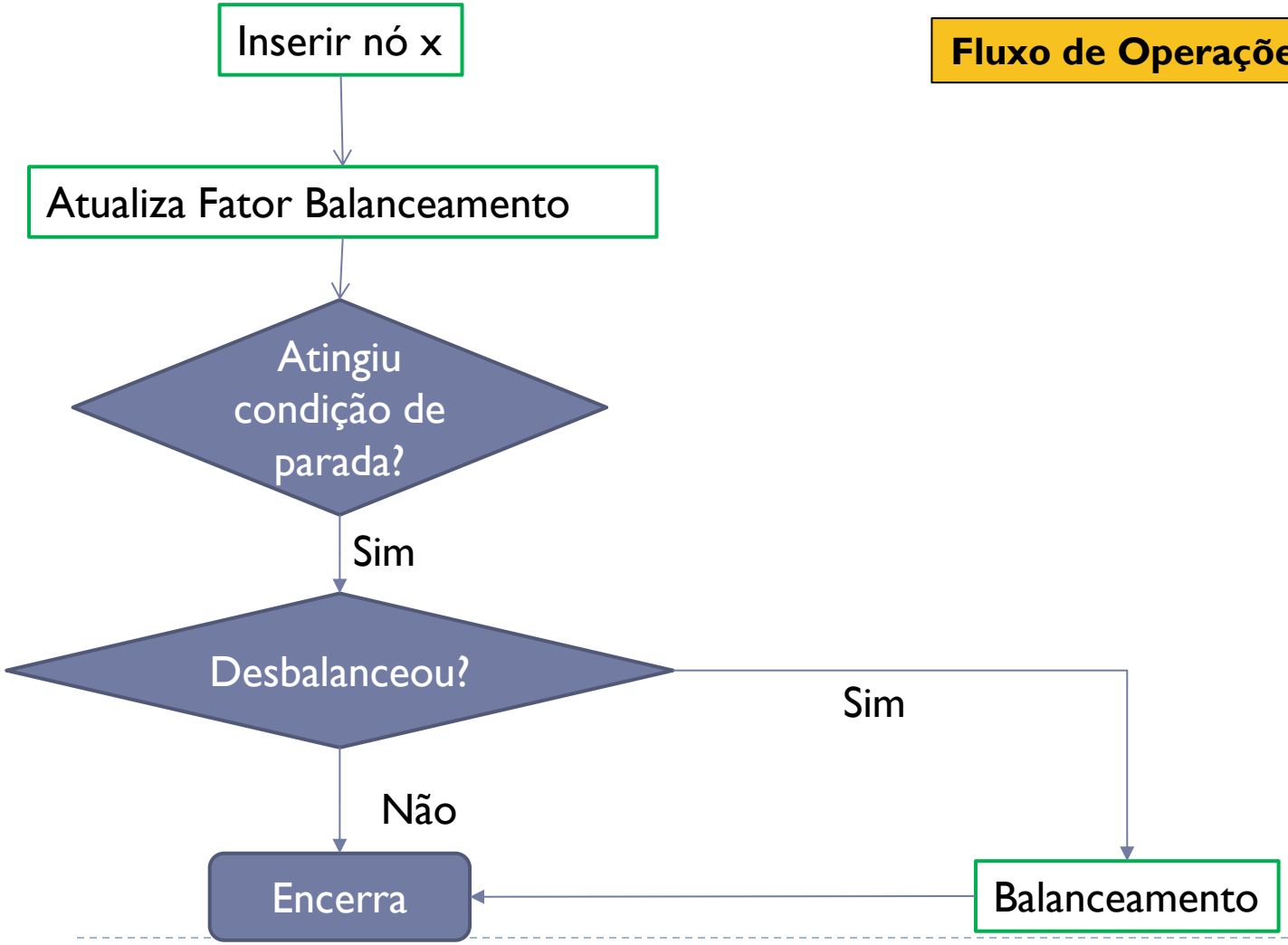
Implementação da AVL

- Faremos a implementação da AVL
  - Iterativa
  - Armazenando o fator de balanceamento

$$fb = h_{dir} - h_{esq}$$

```
struct no{
    int chave;
    int fb;
    struct no *esq;
    struct no *dir;
    struct no *pai;
};
```

Implementação da AVL – INSERÇÃO

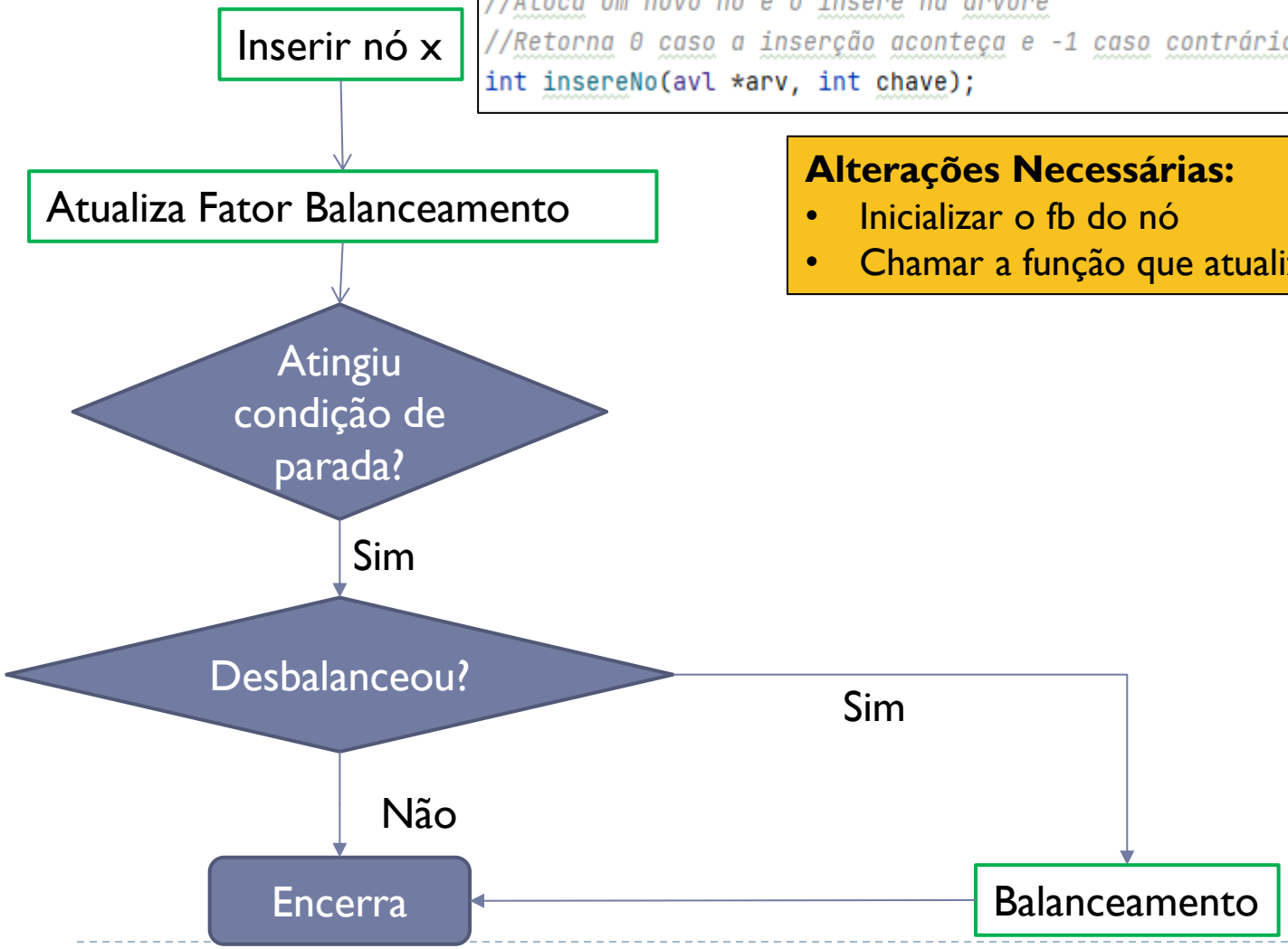


Implementação da AVL – INSERÇÃO

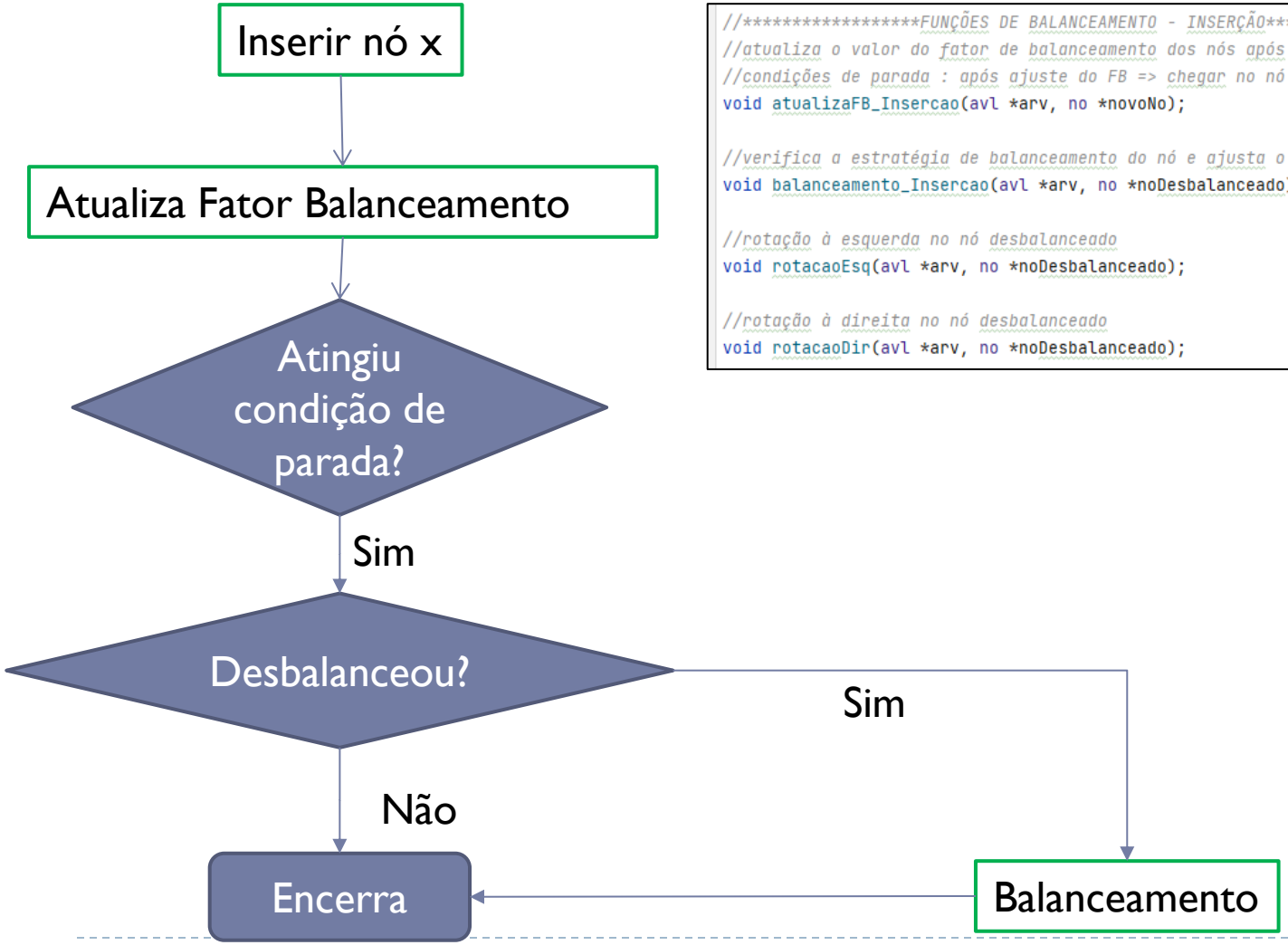
```
//Aloca um novo nó e o insere na árvore
//Retorna 0 caso a inserção aconteça e -1 caso contrário
int insereNo(avl *arv, int chave);
```

Alterações Necessárias:

- Inicializar o fb do nó
- Chamar a função que atualiza o FB



Implementação da AVL – INSERÇÃO




```c
//*****************FUNÇÕES DE BALANCEAMENTO - INSERÇÃO*************************//
//atualiza o valor do fator de balanceamento dos nós após a inserção de um nó
//condições de parada : após ajuste do FB => chegar no nó raiz ou o nó ficar com fb 0, 2 ou -2
void atualizaFB_Insercao(avl *arv, no *novoNo);

//verifica a estratégia de balanceamento do nó e ajusta o fator de balanceamento
void balanceamento_Insercao(avl *arv, no *noDesbalanceado);

//rotação à esquerda no nó desbalanceado
void rotacaoEsq(avl *arv, no *noDesbalanceado);

//rotação à direita no nó desbalanceado
void rotacaoDir(avl *arv, no *noDesbalanceado);
```

Implementação da AVL – INSERÇÃO

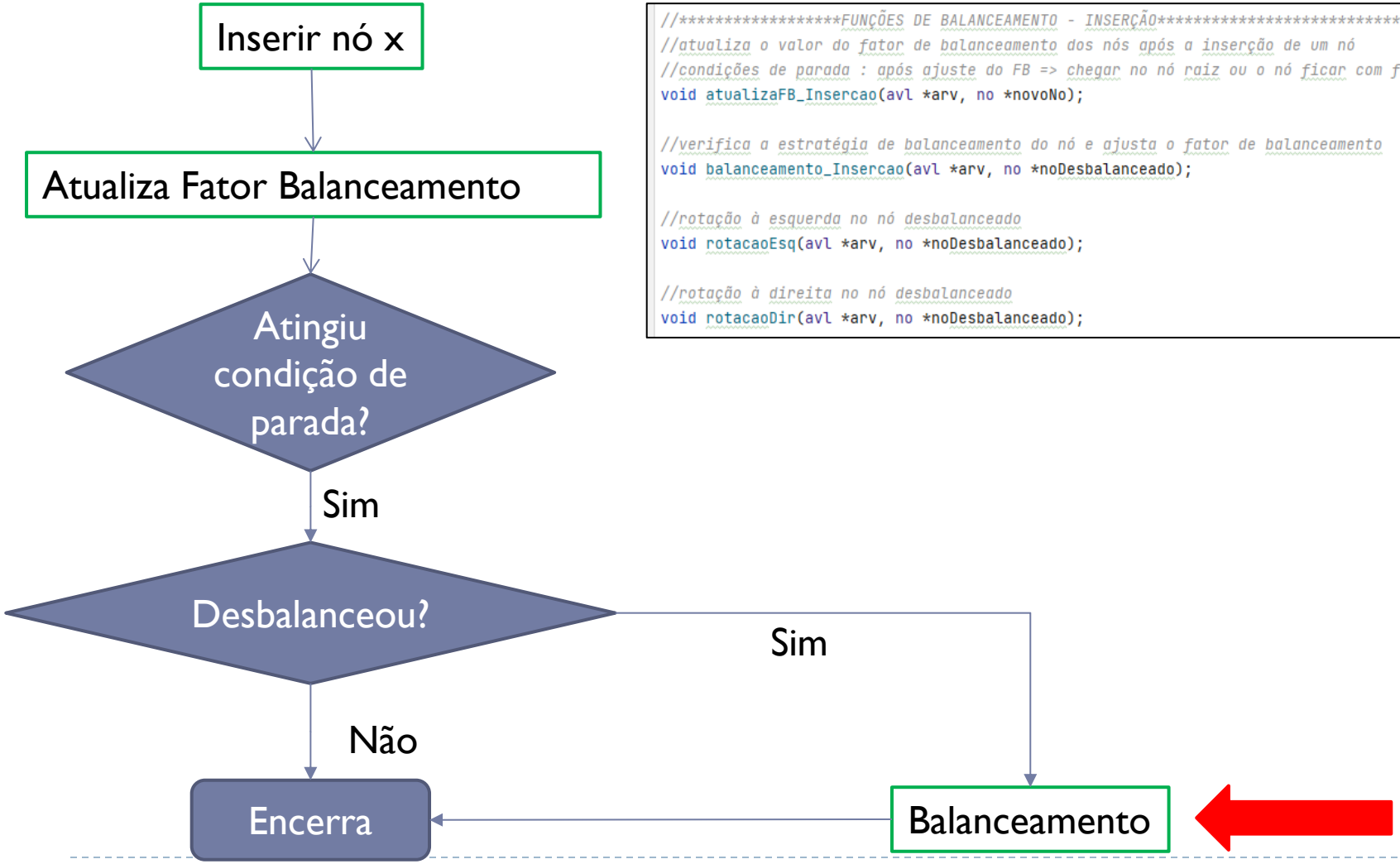
Algoritmo: AVL - ATUALIZAFB-INSERCAO

Entrada: **arv:** Ponteiro para a árvore AVL; **novoNo:** Ponteiro para o nó inserido

Saída: void

```
1  início
2      no *aux ← novoNo // aux inicia no endereço do novo nó
3      do
4          se aux → chave < aux → pai → chave então
5              ???
6          senão
7              ???
8          fim
9          aux ← aux → pai
10     while condições de parada;
11     se desbalanceou? então
12         ???
13     fim
14 fim
```

Implementação da AVL – INSERÇÃO



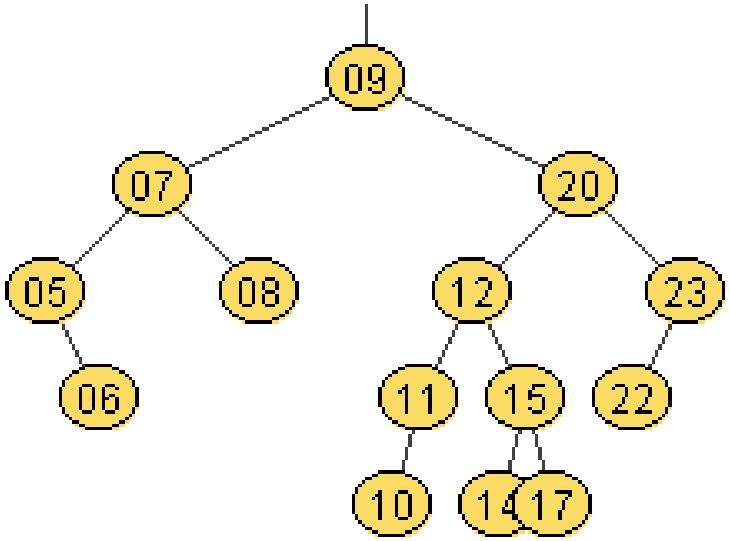

```c
//*****************FUNÇÕES DE BALANCEAMENTO - INSERÇÃO*************************//
//atualiza o valor do fator de balanceamento dos nós após a inserção de um nó
//condições de parada : após ajuste do FB => chegar no nó raiz ou o nó ficar com fb 0, 2 ou -2
void atualizaFB_Insercao(avl *arv, no *novoNo);

//verifica a estratégia de balanceamento do nó e ajusta o fator de balanceamento
void balanceamento_Insercao(avl *arv, no *noDesbalanceado);

//rotação à esquerda no nó desbalanceado
void rotacaoEsq(avl *arv, no *noDesbalanceado);

//rotação à direita no nó desbalanceado
void rotacaoDir(avl *arv, no *noDesbalanceado);
```

Implementação da AVL – INSERÇÃO

Diferença de altura no nó	Diferença de altura do nó filho do nó desbalanceado	Tipo de rotação
2	1	Simples à esquerda
	0	Simples à esquerda
	-1	Dupla : direita – esquerda
-2	1	Dupla : esquerda - direita
	0	Simples à direita
	-1	Simples à direita

Implementação da AVL – INSERÇÃO

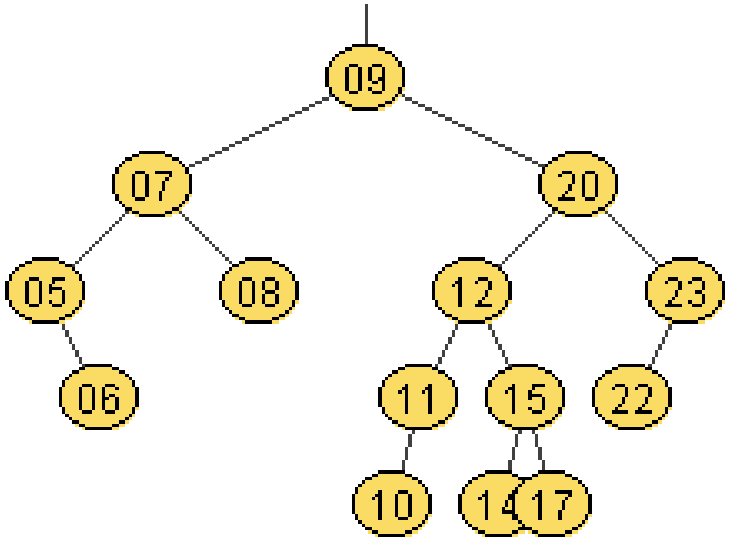
- Após a rotação dupla é necessário atualizar o FB dos nós envolvidos na rotação



Inserir o nó 18

Implementação da AVL – INSERÇÃO

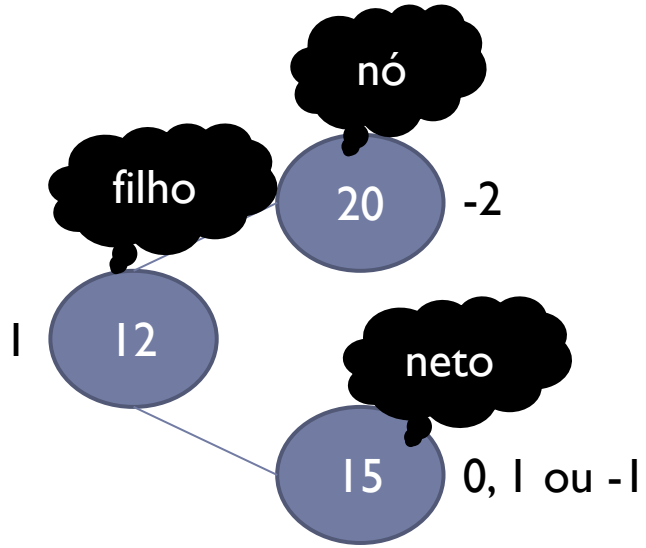
- Após a rotação dupla é necessário atualizar o FB dos nós envolvidos na rotação



Inserir o nó 13

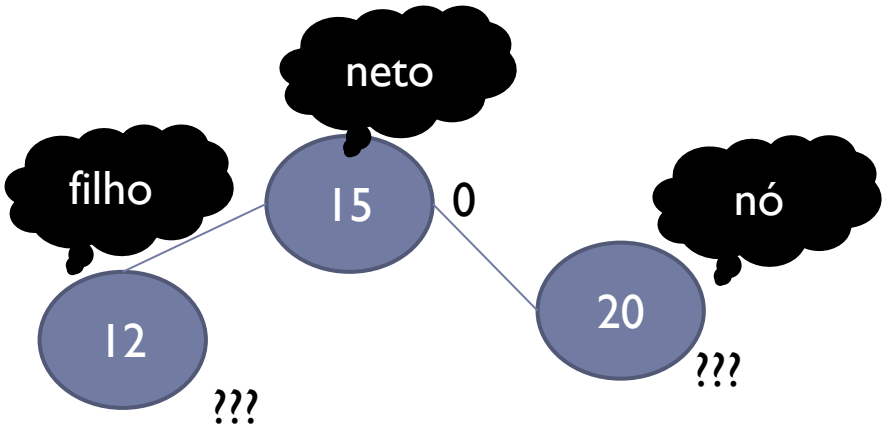
Implementação da AVL – INSERÇÃO

- Após a rotação dupla é necessário atualizar o FB dos nós envolvidos na rotação – Rotação Esquerda-Direita



Implementação da AVL – INSERÇÃO

- Após a rotação dupla é necessário atualizar o FB dos nós envolvidos na rotação – Rotação Esquerda-Direita



Implementação da AVL – INSERÇÃO

- Na rotação dupla Direita/Esquerda
  - Se o balanceamento do neto antes da rotação era 0, todos os nós estarão balanceados no final
  - Se o balanceamento do neto antes da rotação era -1, o balanceamento do filho deve ser atualizado para 1
  - Se o balanceamento do neto antes da rotação era 1, o balanceamento do nó deve ser atualizado para -1

Simétrico
para esq/dir

AVL - Implementação

- Ajuste do Fator de Balanceamento

Antes da Rotação			Rotação	Depois da Rotação		
Nó	Filho	Neto		Nó	Filho	Neto
-2	-1		Simples à Direita	0	0	
-2	0		Simples à Direita	-1	1	
2	1		Simples à Esquerda	0	0	
2	0		Simples à Esquerda	1	-1	
-2	1	0	Dupla Esq-Dir	0	0	0
-2	1	-1		1	0	0
-2	1	1		0	-1	0
2	-1	0	Dupla Dir-Esq	0	0	0
2	-1	-1		0	1	0
2	-1	1		-1	0	0

https://observablehq.com/@infowantstobeseen/avl-trees-understanding-rotations

Implementação da AVL

Algoritmo: AVL - BALANCEAMENTO

Entrada: **arv:** Ponteiro para a árvore AVL; **noDesbalanceado:** Ponteiro para o nó desbalanceado

Saída: void

```
1  início
2      no *filho // ponteiro para o nó filho do nó desbalanceado
3      se noDesbalanceado → fb == 2 então
4          filho ← noDesbalanceado → dir
5          se filho → fb == −1 então
               // Rotação Dupla
6              int fbNeto ← filho → esq → fb // guardar o fb do neto
7              rotacaoDir(arv, filho)
8              rotacaoEsq(arv, noDesbalanceado)
               // ajustar o fb
9          senão
10             int fbFilho ← filho → fb // guardar o fb do filho
11             rotacaoEsq(arv, noDesbalanceado)
12             se fbFilho == 0 então
                   // só acontece na remoção
                   // Ajustar o FB
13             fim
14         fim
15     senão
16     fim
17     ESPELHAR PARA O OUTRO LADO
18 fim
```

Implementação da AVL

```
LEFT-ROTATE(T, x)
y = x.right                    // set y
x.right = y.left               // turn y's left subtree into x's right subtree
if y.left ≠ T.nil
    y.left.p = x
y.p = x.p                      // link x's parent to y
if x.p == T.nil
    T.root = y
elseif x == x.p.left
    x.p.left = y
else x.p.right = y
y.left = x                     // put x on y's left
x.p = y
```

Cormen